

МОСКОВСКИЙ ГОСУДАРСТВЕННЫЙ ТЕХНИЧЕСКИЙ УНИВЕРСИТЕТ
им. Н.Э. Баумана

Факультет “Информатика и системы управления”
Кафедра “Системы обработки информации и управления”



Дисциплина “Парадигмы и конструкции языков программирования”

Отчет по лабораторной работе №5
“Разработка простого бота для Telegram с использованием языка Python”

Выполнил:
Студент группы ИУ5-35Б
Иванов Б. М.
Преподаватель:
Гапанюк Ю. Е.

Москва 2025

Цель лабораторной работы

Цель лабораторной работы: изучение разработки ботов в Telegram.

Задание лабораторной работы

Разработайте простого бота для Telegram. Бот должен использовать функциональность создания кнопок.

Текст программы

Файл Weather.py

```
from requests.exceptions import RequestException
import requests
import time
def retry(func):
    def wrapper_retry(*args, **kwargs):
        seconds = 0.5
        while True:
            try:
                return func(*args, **kwargs)
            except RequestException:
                print(f"Failed to get data from API. Retrying in {seconds} seconds")
                time.sleep(seconds)
        return wrapper_retry
class Weather:
    def __init__(self, *, location:str, date:str):
        base_url = 'https://weather.visualcrossing.com/VisualCrossingWebServices/rest/services/timeline'
        unit_group = 'metric'
        content_type = 'json'
        api_key = "TOKEN" #Weather token
        self.url = f"{base_url}/{location}/{date}/{date}?unitGroup={unit_group}&contentType={content_type}&key={api_key}"
        self.weather_by_hours=[]
        self.get_weather_by_hours_for_day_from_api()
    @retry
    def get_weather_by_hours_for_day_from_api(self):
        response = requests.get(self.url)
        weather_by_days = response.json()["days"]
        self.weather_by_hours = weather_by_days[0]["hours"]
    def get_temperature_hours(self)->dict:
        temp = {}
        for weather in self.weather_by_hours:
            temp[weather["datetime"][:-3]] = weather["temp"]
        return temp
```

Файл temp_graph.py

```
import matplotlib.pyplot as plt
import matplotlib.dates as mdates
from datetime import datetime
def plot_temperature(data:dict,location:str, filename='temperature_plot.png'):
    times = []
    temps = []
    for time_str, temp in data.items():
        time_obj = datetime.strptime(time_str, '%H:%M')
        times.append(time_obj)
        temps.append(temp)
    sorted_data = sorted(zip(times, temps))
    times_sorted, temps_sorted = zip(*sorted_data)
    fig, ax = plt.subplots(figsize=(14, 7))
    ax.plot(times_sorted, temps_sorted, marker='o', linestyle='-',
            color='red', linewidth=3, markersize=8, markerfacecolor='white', markeredgewidth=2)
    ax.fill_between(times_sorted, temps_sorted, alpha=0.3, color='red')
    ax.xaxis.set_major_formatter(mdates.DateFormatter('%H:%M'))
    ax.xaxis.set_major_locator(mdates.HourLocator(interval=3))
    ax.set_title(f'Hourly air temperature in {location}', fontsize=16, fontweight='bold', pad=20)
    ax.set_xlabel('Time', fontsize=12)
    ax.set_ylabel('Temperature (°C)', fontsize=12)
    ax.grid(True, alpha=0.2)
    plt.setp(ax.xaxis.get_majorticklabels(), rotation=45)
    for time, temp in zip(times_sorted, temps_sorted):
        ax.annotate(f'{temp}°C',
                    xy=(time, temp),
                    xytext=(0, 15),
                    textcoords='offset points',
                    ha='center',
                    fontsize=10,
                    bbox=dict(boxstyle="round,pad=0.3", facecolor="yellow", alpha=0.7))
    plt.tight_layout()
    plt.savefig(filename, dpi=300, bbox_inches='tight')
    plt.close()
    print(f'Saved to: {filename}')
```

Файл weather_bot.py

```
from telebot import TeleBot
from telebot.types import ReplyKeyboardMarkup, KeyboardButton
from Weather import Weather
from temp_gpaph import plot_temperature
import datetime
import os
TOKEN = "TOKEN" #TG bot token
LOCATIONS = {"Moscow":"Moscow","Pskov":"Pskov","Udomlya":"Udomlya"}
bot = TeleBot(TOKEN)
@bot.message_handler(commands=["start", "help"])
def send_welcome(message):
    markup = ReplyKeyboardMarkup(row_width=3)
    for name in LOCATIONS.keys():
        item_button = KeyboardButton(name)
        markup.add(item_button)
    bot.send_message(message.chat.id, "Select the location, when you want to know weather ", reply_markup=markup)
@bot.message_handler(func=lambda message: message.text in LOCATIONS.keys())
def send_temp(message):
    photo_path = './images/my_temperature_plot.png'
    my_weather = Weather(location=message.text, date=str(datetime.date.today()))
    plot_temperature(my_weather.get_temperature_hours(), message.text, photo_path)
    try:
        if not os.path.exists(photo_path):
            bot.send_message(message.chat.id, "Location not exists!")
            return
        with open(photo_path, 'rb') as photo:
            bot.send_photo(message.chat.id, photo, caption=f"Here`s the temperature in {message.text} today.")
    except Exception as e:
        bot.send_message(message.chat.id, "Location not exists!")
bot.infinity_polling()
```

Экранные формы с примерами выполнения программы

